A CASE-STUDY BASED REPORT

ON

BOOK RECOMMENDER WITH RATING PREDICTION USING SVD MATRIX FACTORISATION AND COSINE SIMILARITY

submitted in partial fulfilment of the requirements for the Course

MACHINE LEARNING

BACHELOR OF TECHNOLOGY

in COMPUTER SCIENCE AND ENGINEERING

A.Y.: 2026-27

by

Sai Pranavi Naliveni **2520030514**

Jampani Sai Spoorthi **2520030587**

Under the Esteemed guidance of

Dr.Y.Lakshmi Prasanna

Asst.Professor, Dept. of CS&IT

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

INDEX

Abstract

Introduction 4

Background 4
Motivation for the Project 4
Overview of the Work Carried Out 4
Scope and Responsible-Use Statement 4

Problem Statement 5

Case-Study Scenario 5
Central Case-Study Question 5
Formal Problem Definition 5
Objectives that Guide the Study 5
Assumptions and Limitations 5

Dataset and Exploratory Data Analysis

About the Dataset 6
Purpose of EDA 6
Preparation Steps Actually Performed 6
EDA Exploration - Implementation 6
Summary of EDA Findings 6

Algorithm Selection and Implementation 8

Conclusion

References

Signature of the Team members

ABSTRACT

Choosing a book from the very large catalogues of online bookstores and digital libraries is difficult, and popularity-based lists show the same books to every reader. This project presents a machine-learning prototype that predicts the rating a reader would give to a book and recommends the Top-N books for that reader. The system uses the Book-Crossing dataset, which has 1,149,780 ratings, of which 433,671 are explicit scores from 1 to 10. After keeping only users and books with at least five explicit ratings, 141,081 ratings from 13,030 users on 11,234 books remain, so the user-book matrix is 99.9% empty. Each rating is modelled as the global mean plus a damped user bias and a damped book bias, and a Truncated Singular Value Decomposition (SVD) of the remaining residuals captures hidden reading tastes. Cosine similarity between book factors gives similar-book suggestions. On a held-out test set the model obtains an MAE of 1.2152 and an RMSE of 1.5659, compared with 1.2643 and 1.6205 for a user-mean baseline. Precision@10 and Recall@10 are low (0.0019 and 0.0058), which reflects the sparsity of the data. The model is served through a Streamlit web application that accepts a User ID and shows recommendations, predicted ratings and similar books. The output is a data-driven suggestion and not a guarantee that a reader will like a book.

INTRODUCTION

1.1 Background

Recommender systems are machine-learning systems that suggest items to users from the past behaviour of many users. Book recommendation applies the idea to online bookstores and digital libraries, where a reader must choose among hundreds of thousands of titles. Collaborative filtering predicts what a user will like from the ratings of similar users and similar items, and matrix factorisation, including SVD, learns a small number of hidden factors that explain the observed ratings [1], [6], [10].

The literature survey of fifteen papers (2018-2026) shows that collaborative filtering, matrix factorisation and SVD are the most common approaches for book rating prediction. They are usually evaluated with RMSE and MAE, and sometimes with Precision@K and Recall@K [2], [6], [7], [10], [13]. Almost every study reports data sparsity and the cold-start problem as unsolved limitations [4], [6], [10]. Studies also tend to optimise either rating prediction [2] or ranking [3], and deep or graph-based models are costly to deploy [5], [14], [15].

1.2 Motivation for the Project

The motivation is to build a simple, accessible system that gives each reader personal book suggestions together with a predicted rating, instead of the same popular list for everyone. The project connects concepts from data pre-processing, sparse matrices, matrix factorisation, evaluation metrics and web development. It also highlights an important limitation: when 99.9% of the user-book matrix is empty, a model can improve only modestly on simple averages, so honest evaluation against baselines is essential.

1.3 Overview of the Work Carried Out

The system has three parts. First, the Book-Crossing data are cleaned and filtered, and the ratings are split into training, validation and test sets. Second, a bias-aware Truncated SVD model is trained: the number of latent factors is tuned on the validation set ($k = 100$ was best), and the final model is trained on the training plus validation data and evaluated once on the test set against global-mean, user-mean and item-mean baselines. Third, the trained model is saved with pickle and loaded by a Streamlit web application with three tabs: recommendations for a User ID, rating prediction for a user and an ISBN, and similar books using cosine similarity. The work was developed and run in Google Colab using Python.

1.4 Scope and Responsible-Use Statement

The scope is limited to an academic prototype that recommends books from explicit rating data. The system does not use book content (text, genre, cover), user demographics or the time of rating, and it cannot personalise for a new user or a new book that has no ratings in the training data; for such users the application shows popular books. The dataset was collected in 2004, so the recommendations reflect the tastes of that period. Predicted ratings and recommendations are suggestions only and may be wrong; they must not be treated as a guarantee that a reader will enjoy a book. User IDs in the dataset are anonymised numbers and no personal details are used.

PROBLEM STATEMENT

2.1 Case-Study Scenario

A reader on an online bookstore faces hundreds of thousands of titles. A page that lists the most popular or highest-average books is the same for every reader and ignores their rating history. The reader would like a short list of books that match their own taste, and an estimate of the rating they would give a book before spending time reading it.

2.2 Central Case-Study Question

Can a matrix-factorisation model trained on past user ratings predict how a user would rate an unseen book and rank the books that the user is most likely to enjoy, better than simple average-based predictions, even when almost all of the user-book matrix is empty?

2.3 Formal Problem Definition

Let U be the set of users and I the set of books. The observed data are triples (u, i, r_ui), where r_ui is an integer rating from 1 to 10 given by user u to book i. The task has two parts: (a) rating prediction, which learns a function r^(u, i) that estimates the rating for an unseen pair; and (b) Top-N recommendation, which returns for a user u the N books not yet rated by u that have the highest predicted rating. The model used is r^(u, i) = mu + b_u + b_i + p_u . q_i, clipped to the range [1, 10], where mu is the global mean rating, b_u and b_i are the user and book biases, and p_u and q_i are the k-dimensional latent factor vectors obtained from Truncated SVD.

2.4 Objectives that Guide the Study

The following objectives guide the study:

- To collect and prepare the Book-Crossing data (1,149,780 ratings) and keep explicit ratings from users and books with at least 5 ratings (141,081 ratings).
- To design and implement a bias-aware SVD matrix-factorisation model, with the number of latent factors tuned on a validation set, that predicts ratings.
- To add cosine similarity between book factors for similar-book suggestions.
- To evaluate the model on a held-out test set using MAE and RMSE, to compare it with global-mean, user-mean and item-mean baselines, and to measure Precision@K and Recall@K of the Top-N list.
- To deploy a Streamlit web application that takes a User ID and shows Top-N books with predicted ratings.
- To discuss the limitations of the approach, mainly data sparsity and the cold-start problem.

2.5 Assumptions and Limitations

The project assumes that explicit ratings from 1 to 10 reflect real preferences and that a rating of 0 means that no score was given, so zero ratings are excluded. Users and books with fewer than five ratings are removed because too little is known about them, which also means that new users and new books cannot be personalised. The random 70/10/20 split (seed 42) assumes that test ratings are similar to training ratings. The ratings are very sparse and skewed towards popular books, so the model improves on simple averages only modestly (Section 4.6), and Precision@10 and Recall@10 are low. A relevant book in the test set is defined as one rated 7 or more by the user, and each user has only a few such books, which keeps ranking metrics small. The dataset dates from 2004 and may not represent current readers.

3. DATASET AND EXPLORATORY DATA ANALYSIS

3.1 About the Dataset

The project uses the Book-Crossing dataset, collected by Cai-Nicolas Ziegler in 2004 from the Book-Crossing community [16]. It has three files: Books.csv (ISBN, Book-Title, Book-Author, Year-Of-Publication, Publisher and image URLs), Ratings.csv (User-ID, ISBN, Book-Rating) and Users.csv (User-ID, Location, Age). The Ratings file contains 1,149,780 ratings. A rating of 0 means that the user interacted with the book but gave no score (implicit), and ratings from 1 to 10 are explicit scores. User IDs are anonymised numbers. The dataset is used for model development and is not a complete record of everything people read.

3.2 Purpose of EDA

Exploratory Data Analysis (EDA) was used to understand the size of the data, the distribution of ratings, the number of users and books, the most-rated books and the sparsity of the user-book matrix before modelling. It showed that most ratings are 0 (no score) and that explicit ratings are concentrated at high values, so zero ratings must be removed and the average rating (7.6) must be handled with bias terms.

3.3 Preparation Steps

- Load Books.csv, Ratings.csv and Users.csv with pandas (comma separator, latin-1 encoding, bad lines skipped) and strip the column names.
- Convert User-ID and Book-Rating to numbers, trim the ISBN strings, drop rows with missing values and keep ratings between 0 and 10; no rows were lost (1,149,780 ratings remain).
- Keep only explicit ratings (1 to 10): 433,671 ratings remain.
- Remove duplicate (User-ID, ISBN) pairs, keeping the last rating.
- Keep users with at least 5 ratings, then books with at least 5 ratings: 141,081 ratings remain from 13,030 users and 11,234 books.
- Split the data randomly (seed 42) into 20% test data, then split the remaining 80% into 87.5% training and 12.5% validation data, which gives about 70% / 10% / 20%.
- Estimate the biases and fit the SVD on the training data only; the test set is used once for the final evaluation.

3.4 EDA Exploration and Implementation

The following code from the project notebook performs the EDA and the filtering.

```
Ratings["Book-Rating"] = pd.to_numeric(Ratings["Book-Rating"], errors="coerce")
ratings_clean = Ratings.dropna(subset=["User-ID", "ISBN", "Book-Rating"])
ratings_explicit = ratings_clean[ratings_clean["Book-Rating"] > 0]
print(len(ratings_clean), len(ratings_explicit))          # 1149780  433671
print(ratings_explicit["Book-Rating"].mean())             # 7.6

ml_data = ratings_explicit[["User-ID", "ISBN", "Book-Rating"]]
ml_data = ml_data.drop_duplicates(subset=["User-ID", "ISBN"], keep="last")
ml_data = ml_data[ml_data["User-ID"].isin(user_counts[user_counts >= 5].index)]
ml_data = ml_data[ml_data["ISBN"].isin(book_counts[book_counts >= 5].index)]
```

3.5 Summary of EDA Findings

The EDA findings are summarised below. Most of the 1,149,780 ratings are zeros, which carry no score, so only 433,671 explicit ratings (mean 7.6) are used. Explicit ratings are concentrated between 7 and 10 (Figure 1), so a model that predicts a value near the mean already does reasonably well; this is why user

and book biases matter. After filtering, the user-book matrix has 13,030 rows and 11,234 columns but only 141,081 filled cells, so it is 99.9% empty.

Table 1: Summary of the dataset before and after filtering

Measure	Value
Total ratings in Ratings.csv	1,149,780
Ratings with score 0 (no explicit score)	716,109 (62.3%)
Explicit ratings (1 to 10)	433,671
Mean explicit rating	7.6
Distinct users / books in Ratings.csv	105,283 / 340,553
After filtering (5+ ratings per user and book): ratings	141,081
After filtering: users x books	13,030 x 11,234
Sparsity of the user-book matrix	99.9%

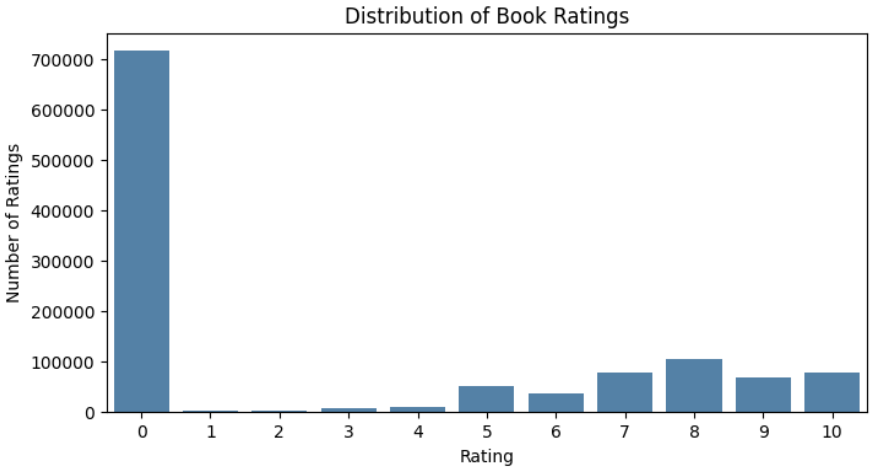


Figure 1: Distribution of book ratings in Book-Crossing (0 = no explicit score)

The ten most rated books are shown in Table 2. The most popular book has only 707 explicit ratings, which shows how thinly the ratings are spread over the books.

Table 2: Ten most rated books (explicit ratings)

ISBN	Book title	Explicit ratings
0316666343	The Lovely Bones: A Novel	707
0971880107	Wild Animus	581
0385504209	The Da Vinci Code	487
0312195516	The Red Tent (Bestselling Backlist)	383
0679781587	(title not available in Books.csv)	333
0060928336	Divine Secrets of the Ya-Ya Sisterhood: A Novel	320
059035342X	Harry Potter and the Sorcerer's Stone	313
0142001740	The Secret Life of Bees	307
0446672211	Where the Heart Is (Oprah's Book Club)	295
044023722X	A Painted House	281

ALGORITHM SELECTION AND IMPLEMENTATION

4.1 Bias Estimation and Residual Matrix

Before factorisation, the average behaviour of users and books is removed. The global mean mu is the mean training rating. The book bias is b_i = sum(r_ui - mu) / (n_i + lambda), and after removing mu and b_i the user bias is b_u = sum(r_ui - mu - b_i) / (n_u + lambda), where n_i and n_u are the numbers of ratings of the book and of the user, and lambda = 10 is a damping term that shrinks the biases of books and users with few ratings towards zero. Without damping, one 10-star rating would make a book look perfect.

The residual of each known rating is e_ui = r_ui - mu - b_i - b_u, and the residuals are stored in a sparse CSR matrix (users x books). Unrated cells are treated as zero residual, which means 'no information beyond the averages'. This is the key difference from the first version of the project, which filled missing ratings with 0 and therefore gave an MAE of 6.76 and an RMSE of 6.99, because the model learned that unrated books were rated 0.

4.2 Algorithm: Truncated SVD and Cosine Similarity

Truncated SVD (scikit-learn TruncatedSVD) factorises the sparse residual matrix E into U, Sigma and V^T with k components. The user factors P = U x Sigma (users x k) and the book factors Q = V^T (k x books) are saved. The predicted rating is r^(u, i) = mu + b_u + b_i + P_u . Q_i, clipped to the range 1 to 10. SVD was chosen because it works on very sparse matrices, is fast, and is widely used for book rating prediction [6], [8], [10].

Cosine similarity between two books i and j is cos(i, j) = (Q_i . Q_j) / (|Q_i| |Q_j|); a value near 1 means that the two books appeal to similar readers. To produce Top-N recommendations for a user, the model predicts a rating for every book, removes the books the user has already rated and sorts the rest from the highest to the lowest predicted rating.

4.3 Training and Evaluation

The data were split 70/10/20 into training, validation and test sets (seed 42), which gives about 98,756 training, 14,108 validation and 28,217 test ratings. The number of latent factors k was selected on the validation set from {10, 20, 50, 100} using RMSE (Table 3). The lowest validation RMSE was obtained with k = 100, although the differences are very small. The final model was then trained on the training plus validation data (112,864 ratings) and evaluated once on the test set.

Table 3: Validation error for different numbers of latent factors

Latent factors k	Validation MAE	Validation RMSE
10	1.220	1.572
20	1.220	1.572
50	1.220	1.571
100 (selected)	1.219	1.570

Rating accuracy is measured with MAE (the average absolute error) and RMSE (the square root of the average squared error, which penalises large errors more). The quality of the Top-N list is measured with Precision@K and Recall@K on all test users who are in the model and have at least one relevant test book, where a relevant book is one the user rated 7 or higher (7,324 users). Precision@K is the fraction of the K recommended books that are relevant, and Recall@K is the fraction of the user's relevant books that appear in the K recommendations.

4.4 Web Application and Request Flow

- The front end is a Streamlit web page written in Python. A sidebar chooses the number of recommendations N (1 to 20), and a panel shows the model metrics (MAE, RMSE, Precision@10, Recall@10).
- Tab 1 (Recommend for user): the user enters a User ID. The app loads the saved files model_factors.pkl, seen_by_user.pkl, metrics.pkl and book_info.csv.
- If the ID belongs to a trained user (a user with at least five ratings in the cleaned data), the app predicts ratings for all books, removes books the user has already rated and shows the Top-N books with title, author, ISBN and predicted rating. For an unknown ID it shows popular books with the highest book bias (cold-start fallback).
- Tab 2 (Predict a rating): the user enters a User ID and an ISBN, and the app returns mu + b_u + b_i + P_u . Q_i clipped to 1 to 10.
- Tab 3 (Similar books): the user enters an ISBN, and the app returns the books with the highest cosine similarity.
- Predictions are computed on demand from the saved factors, so the large dense prediction matrix (about 1.1 GB in the first version) is no longer stored; the saved model is about 20 MB.

4.5 Running the Project in Google Colab

The project runs in Google Colab. Open the notebook and run the cells in order: install the libraries (pandas, numpy, scipy, scikit-learn, matplotlib, seaborn, streamlit); upload Books.csv, Ratings.csv and Users.csv; run the loading, EDA and preparation cells; run the model cell, which tunes k, evaluates the model and saves model_factors.pkl, seen_by_user.pkl, metrics.pkl and book_info.csv; write app.py with %%writefile app.py; and run the launch cell, which starts Streamlit on port 8501 and prints a Colab link. In the app, enter a User ID from the trained users, for example one printed by list(model['u_idx'])[:5]. The libraries are open source and no GPU is needed.

4.6 Results and Discussion

The bias-aware SVD gives the lowest MAE and RMSE on the test set (Table 4). Compared with the global mean it reduces MAE by 14.1% and RMSE by 11.8%; compared with the user-mean baseline the reduction is smaller, 3.9% for MAE and 3.4% for RMSE. Most of the improvement therefore comes from the user and book biases, and the latent factors add a small further gain, which is also seen in the validation results where changing k barely changes the error. The first version of the project, which treated unrated cells as zero, had an MAE of 6.76 and an RMSE of 6.99.

Table 4: Rating prediction error on the test set

Model	Test MAE	Test RMSE
Global mean	1.4146	1.7754
User mean (mu + b_u)	1.2643	1.6205
Item mean (mu + b_i)	1.3675	1.7235
Bias-aware SVD (k = 100)	1.2152	1.5659

Table 5: Top-N quality on the test set (relevant = rated 7 or higher)

K	Precision@K	Recall@K	Users evaluated
5	0.0026	0.0036	7,324
10	0.0019	0.0058	7,324

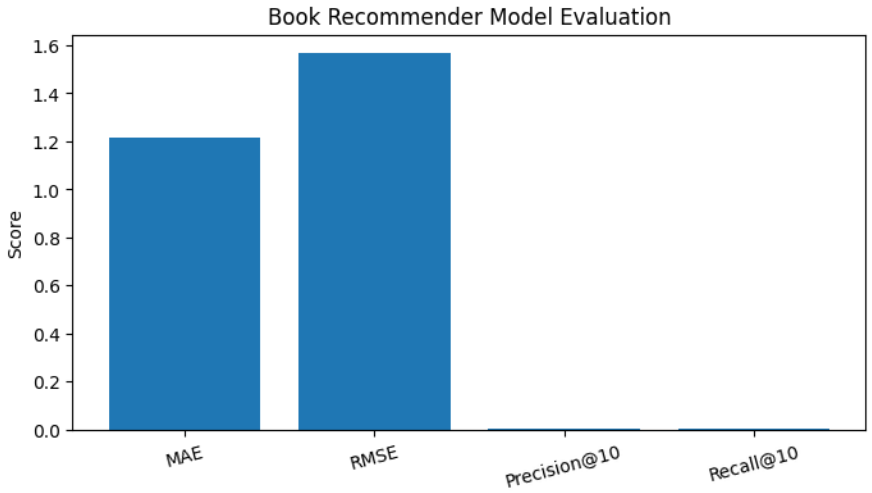


Figure 2: Model evaluation scores (MAE, RMSE, Precision@10, Recall@10)

Precision@10 of 0.0019 and Recall@10 of 0.0058 are low: on average about one in five hundred recommended books was a relevant book in the test set. This is expected with a matrix that is 99.9% empty, because each user has only a few relevant test books, and ranking by predicted rating pushes widely loved popular books to the top. The result has not yet been compared with a 'most popular books' recommender, which would show how much personalisation adds; this is the first item of future work. Table 6 shows the Top-10 recommendations for an example user from the training data; the list is plausible (fantasy and children's classics), which suggests that the model captures broad tastes.

Table 6: Top-10 recommendations for an example user

Rank	Book title	Author	Predicted rating
1	Harry Potter and the Chamber of Secrets	J. K. Rowling	9.23
2	The Return of the King	J.R.R. Tolkien	9.21
3	Harry Potter and the Goblet of Fire	J. K. Rowling	9.13
4	The Two Towers	J.R.R. Tolkien	9.07
5	Harry Potter and the Sorcerer's Stone	J. K. Rowling	9.03
6	The Cat in the Hat	Dr. Seuss	9.01
7	Calvin and Hobbes	Bill Watterson	9.01
8	The Hobbit	J.R.R. Tolkien	8.99
9	The Authoritative Calvin and Hobbes	Bill Watterson	8.97
10	Where the Sidewalk Ends	Shel Silverstein	8.97

CONCLUSION

This case study demonstrates how a rating dataset, careful filtering, bias estimation, Truncated SVD and a web interface can be combined into a working book recommender. The bias-aware SVD model predicts ratings with an MAE of 1.2152 and an RMSE of 1.5659 on held-out data, which is better than global-mean, user-mean and item-mean baselines, and it provides Top-N recommendations, similar books and predicted ratings through a Streamlit application. The improvement over the user-mean baseline is modest and the Top-N precision and recall are low, which shows the difficulty of very sparse data and the cold-start problem. The next steps are to compare the Top-N list with a most-popular-books baseline, to try item-based or hybrid models that use book content, and to handle new users and new books better.

REFERENCES

[1] H. Alharthi, D. Inkpen and S. Szpakowicz, "A survey of book recommender systems," J. Intell. Inf. Syst., vol. 51, 2018.

[2] K. Davagdorj, D. Li, K. H. Park and K. H. Ryu, "A matrix factorization approach for user rating prediction in book recommender system," 2019.

[3] F. Bahiransyah, D. Nurjanah and R. Rismala, "Bayesian personalized ranking - matrix factorization untuk rekomendasi buku," 2018, doi: 10.34818/eoe.v5i3.7283.

[4] Du et al., "Research on personalized book recommendation based on improved similarity calculation and data filling collaborative filtering algorithm," 2022.

[5] Dien et al., "An approach for learning resource recommendation using deep matrix factorization," 2022.

[6] M. Ahmed, "Book recommendation using collaborative filtering algorithm," Appl. Comput. Intell. Soft Comput., 2023, doi: 10.1155/2023/1514801.

[7] C. Tegetmeier, A. Johannssen and N. Chukhrova, "Artificial intelligence algorithms for collaborative book recommender systems," Ann. Data Sci., 2023, doi: 10.1007/s40745-023-00474-4.

[8] M. Attalariq and Z. K. A. Baizal, "Chatbot-based book recommender system using singular value decomposition," J. Inf. Syst. Res., 2023, doi: 10.47065/josh.v4i4.3817.

[9] R. Akbar, D. Richasdy and R. Dharayani, "Sistem rekomendasi buku dengan collaborative filtering menggunakan metode singular value decomposition (SVD)," 2023.

[10] I. Ishak, A. Yahya and Yusri, "Book recommendation system based on collaborative filtering: user-based, item-based, and singular value decomposition analysis," J. Syst. Comput. Eng., 2025/2026, doi: 10.61628/jsce.v6i4.2201.

[11] P. Luekhong and W. Chansanam, "Advancing book recommendation systems: a comparative analysis of collaborative filtering and matrix factorization algorithms," 2025, doi: 10.52783/jisem.v10i4s.492.

[12] A. Miriyala, P. Chikondra, S. Alwala and N. Anjum, "Book recommendation system using collaborative filtering," Int. J. Novel Res. Dev., 2025.

[13] U. Hasanah, "Book recommendation system using similarity-based collaborative filtering approach," J. Inf. Syst. Artif. Intell., 2026, doi: 10.26486/jisai.v7i2.297.

[14] Wan et al., "A recommendation approach based on heterogeneous network and dynamic knowledge graph," 2024.

[15] Yang et al., "Graph convolutional recommendation system based on bilateral attention mechanism," 2024.

[16] C.-N. Ziegler, S. M. McNee, J. A. Konstan and G. Lausen, "Improving recommendation lists through topic diversification," in Proc. 14th Int. World Wide Web Conf. (WWW), Chiba, Japan, 2005, pp. 22-32 (Book-Crossing dataset).

[17] scikit-learn documentation, TruncatedSVD and model selection (train_test_split).

[18] Streamlit documentation, building and running data applications in Python.

[19] pandas documentation, reading CSV files and group-by operations.

Signature of the Team Members: _____